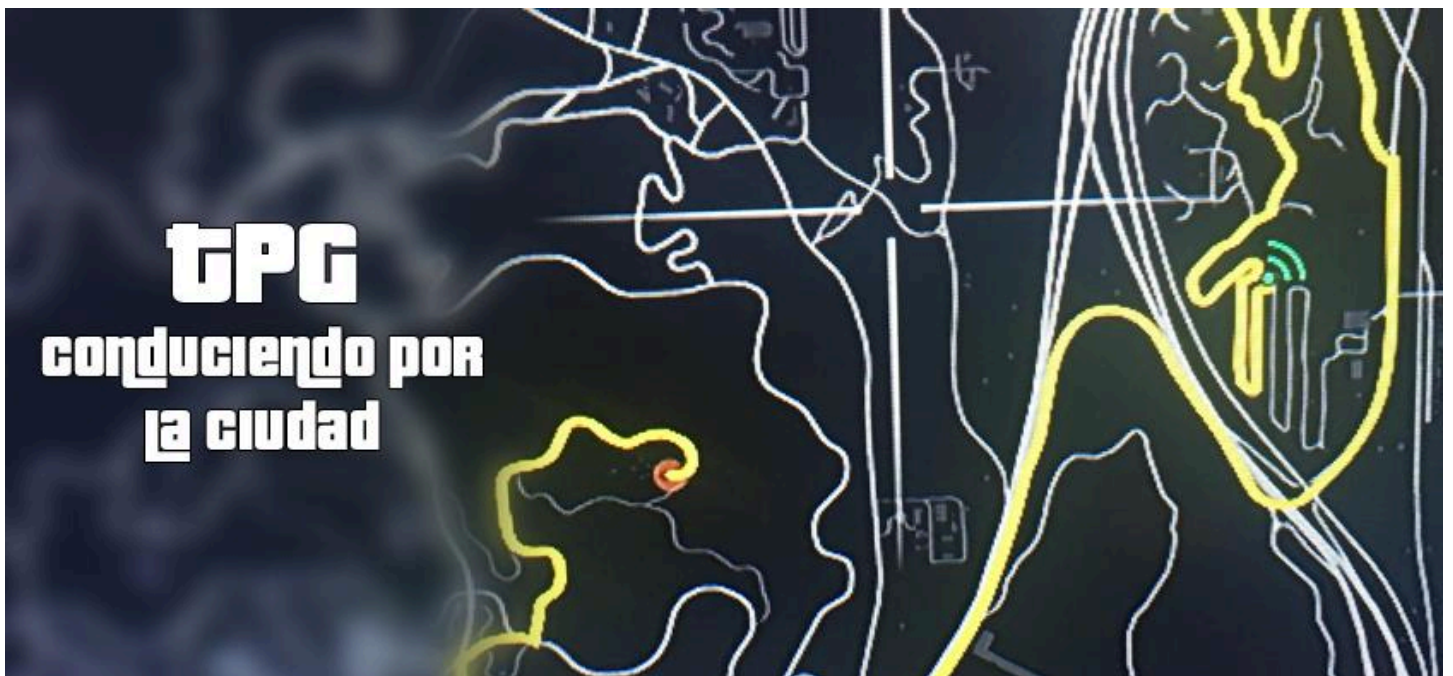




La saga **Grand Theft Auto**, desarrollada por **Rockstar Games**, se ha consolidado como una de las franquicias más influyentes y revolucionarias en la historia de los videojuegos. Desde su primera entrega en 1997, la serie se ha caracterizado por ofrecer mundos abiertos con muchos grados de libertad, historias cargadas de sátira social y una jugabilidad que combina acción, conducción y exploración en entornos urbanos inspirados en la realidad.

Tras el rotundo éxito de *GTA V* (2013), que se convirtió en uno de los juegos más vendidos de todos los tiempos, la expectativa por la siguiente entrega, *GTA VI*, solamente creció desde entonces.

12 años después, **Rockstar** aún no publica este juego. Su desarrollo estuvo caracterizado por el silencio y las demoras, a punto tal que se cuestiona su existencia. Algunos incluso hacen chistes del tipo “*We got X before **GTA VI***”.

Por este motivo, los estudiantes de Algoritmos y Estructuras de Datos *decidieron tomar cartas en el asunto*, ayudando a **Rockstar** a completar el desarrollo del juego mediante la implementación de algunos sistemas.

En esta ocasión, los asignaron a un grupo de trabajo para que, con otros desarrolladores, sigan avanzando en la implementación de **funcionalidades clave** para el juego.

Cambios en vehículos

Para el desarrollo del juego principal, es necesario introducir algunos cambios en los vehículos.

Rockstar solicitó la creación de un tipo de vehículo nuevo: **Exótico**. Los vehículos exóticos son vehículos raros que el jugador podrá conseguir únicamente completando misiones.

Como algunos vehículos exóticos pueden ser vehículos concepto (que no se comercializan de forma masiva), la clase debe admitir una cantidad *variable* de ruedas.

El costo de mantenimiento de un vehículo exótico se calcula de forma similar al de otros vehículos, con excepción del costo por rueda: cada rueda cuesta 10, más 10 por cada rueda anterior. Por ejemplo, si un vehículo tiene 5 ruedas, entonces el costo es:

$$\$10 + \$20 + \$30 + \$40 + \$50 = \$150$$

Además, se deberá agregar un kilometraje a los vehículos (se desarrollará más adelante en el juego principal) y una velocidad máxima.

El kilometraje de un vehículo aumentará su costo de mantenimiento diario:

1. Para autos, el costo aumenta en \$10 por cada 1000 kilómetros.
2. Para motos, el costo aumenta en \$10 por cada 500 kilómetros.
3. Para exóticos, el costo no aumenta por el kilometraje. En cambio, se agrega una cantidad fija de \$100.

También, se deberá permitir cargar el tanque de gasolina de un vehículo. Esto puede ser:

1. Cargar una cantidad específica.
2. Cargar hasta la capacidad máxima.

Cada litro cuesta \$1.

Finalmente, como se espera que haya más tipos de vehículos, **Rockstar** solicitó que los vehículos se implementen con un esquema de *herencia*.

Nueva funcionalidad: el Concesionario

Se deberá desarrollar un concesionario de vehículos para que el jugador pueda comprar vehículos utilizando el dinero que ganó de completar misiones.

El jugador podrá comprar todo tipo de vehículos, excepto vehículos exóticos, ya que estos se ganan únicamente completando misiones.

Dentro del concesionario, el jugador debe poder:

1. Buscar vehículos por nombre
2. Buscar vehículos por marca
3. Comprar un vehículo, indicando el nombre exacto

Para que sea fácil de usar, las primeras dos opciones deben buscar y mostrar los vehículos que cumplan parcialmente con lo ingresado, por ejemplo:

- Si el jugador busca el nombre “Peugeot 30” se mostrarían “Peugeot 306”, “Peugeot 307” y “Peugeot 308”
- Si el jugador busca la marca “Lambo” se mostrarían todos los autos de la marca “Lamborghini”

El concesionario debe ser *flexible* para cualquier vehículo que se quiera vender. Sin embargo, los vehículos que se venden, junto con sus características, pueden estar preestablecidos. Por ejemplo, podría crearse un conjunto de autos *hardcodeados* para luego ser cargados en el concesionario. También se podrían cargar desde un archivo (por ejemplo, en formato **JSON** o **xml**).

Cambios en el Garaje

El garaje deberá ser modificado para soportar las nuevas funcionalidades anteriores.

Principalmente, debe soportar el almacenamiento *temporal* de vehículos comprados desde el concesionario, en el caso de que no haya más espacio disponible.

Para resolver esto, el Garaje deberá implementar una “zona de espera” donde se cargarán los vehículos que se compren desde el concesionario pero que no tengan un espacio disponible en el garaje. Cuando el jugador compre un espacio nuevo de garaje, se intentará ubicar los vehículos de la zona de espera **en el orden en el que fueron comprados**. Es decir, los vehículos que fueron comprados primero serán los primeros en ser ubicados.

Los vehículos en la zona de espera **no pueden ser utilizados para el juego principal**.

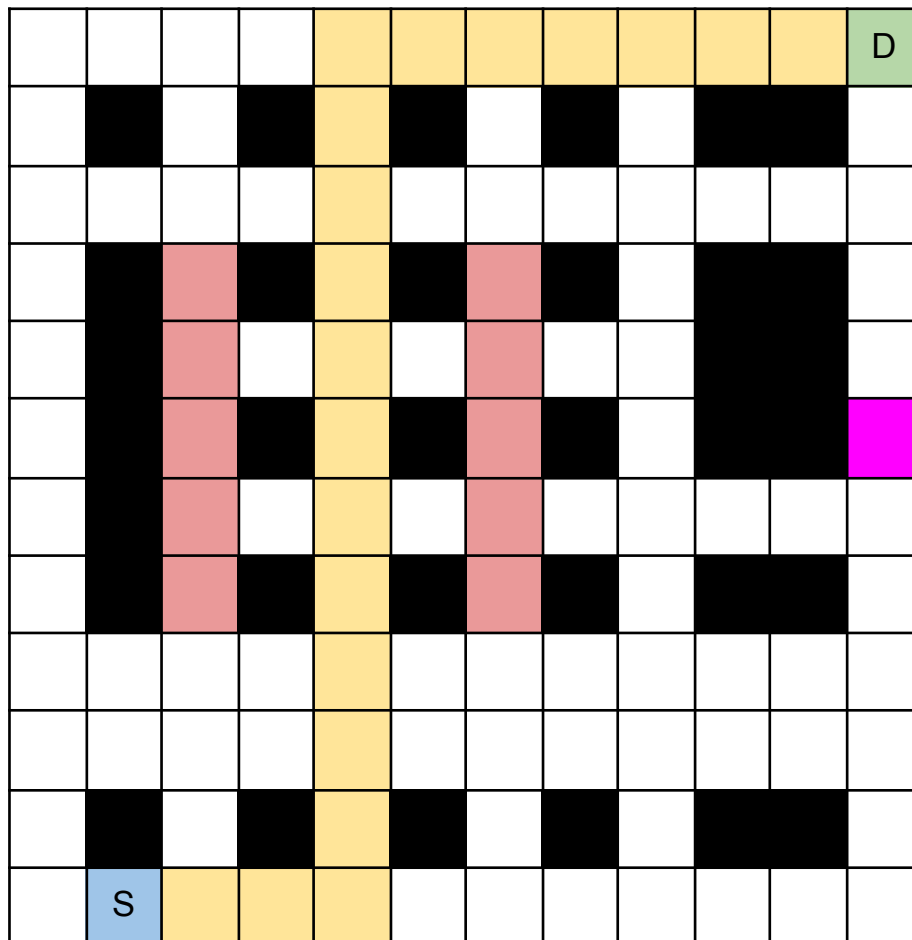
También se debe permitir cargar de forma masiva todos los tanques de gasolina de todos los vehículos, hasta el máximo de cada vehículo (se desarrollará más adelante en el juego principal).

Finalmente, el Garaje deberá modificar el esquema de guardado de su información en formato **.csv**, ajustándose a los cambios de los vehículos, a la zona de espera, y permitiendo que se guarde en conjunto con la información de otras partes del juego, como la información del jugador en el juego principal.

Queda a decisión del grupo de trabajo cómo modificar el formato del archivo.

Juego principal: las Misiones

Se desarrollará un juego principal que utilice todas las funcionalidades anteriores. Este juego girará alrededor de las **Misiones** de conducción alrededor de la ciudad.



Glosario:

1. Blanco: Calle transitable
2. Negro: Edificio
3. Rojo: Congestión
4. Azul: Salida
5. Verde: Destino
6. Amarillo: Camino GPS
7. Magenta: Recompensa extra

El objetivo principal será conducir desde un punto de salida (S) hasta un destino (D), en la menor cantidad de tiempo posible.

El jugador podrá manejar por el mapa de una ciudad, utilizando las teclas **WASD**. Cada movimiento que el jugador realice sumará tiempo al contador, descontará un litro de gasolina del vehículo y agregará un kilómetro al kilometraje:

1. El tiempo de tránsito de cada calle se calcula a partir del costo de la calle (dependiendo de si está congestionada o no), dividido por la **velocidad máxima** del vehículo (vehículos más rápidos tardan menos tiempo).
2. Si el jugador se queda sin gasolina o se queda sin tiempo, la misión falla automáticamente.

El mapa tendrá calles transitables y edificios. El punto de salida y el destino se generan aleatoriamente en cada misión. Ambos puntos deben ser transitables.

Además, de forma aleatoria, el mapa puede tener puntos de congestión, que generan un retardo del **quíntuple** de tiempo en comparación al tránsito normal.

También pueden aparecer recompensas extra, que pueden ser coleccionadas por el jugador. Será decisión del jugador tomar desvíos del mejor camino posible para agarrarlos. Estas recompensas pueden ser créditos para el garaje o, de forma muy rara, un vehículo exótico. Se debe completar la misión para que estas recompensas sean ganadas.

Finalmente, en todo momento, el mapa contará con un GPS que deberá calcular el **mejor camino posible** para llegar desde la celda actual en la que se esté hasta el destino. Este camino **debe actualizarse** en cada movimiento del jugador.

El mapa de la ciudad debe cumplir las siguientes restricciones:

1. Debe ser variable (no *hardcodeado*)
2. Debe tener forma rectangular (debe ser representable mediante una matriz).
3. Debe ser cargado mediante un archivo (el formato del archivo queda a disposición del grupo).

Las misiones tendrán tres dificultades:

1. Fácil: 1000 segundos de tiempo límite, \$1000 y 10 créditos de garaje de recompensa
2. Moderada: 500 segundos de límite de tiempo, \$2500 y 20 créditos de garaje de recompensa

3. Difícil: tipo de vehículo limitado, 250 segundos de límite de tiempo, \$7500 y 50 créditos de garaje de recompensa

(Nota: pueden modificar estos valores para que el juego esté balanceado.)

Las misiones fáciles y moderadas pueden ser completadas con cualquier vehículo. Luego de seleccionar una misión, se debe permitir seleccionar un vehículo del garaje para completarla.

Las misiones difíciles tendrán un tipo de vehículo con el que deben completarse (generado aleatoriamente). Seleccionar una misión difícil que debe completarse utilizando, por ejemplo, un **auto**, debe mostrar todos los **autos** disponibles en el garaje.

En caso de no haber vehículos de ese tipo (o de no tener ningún vehículo), no se debe permitir continuar a la misión.

El juego (*loop*) principal consistirá en jugar una misión, completarla o fallar, y avanzar al siguiente día. Cada día, se descontará el costo total de mantenimiento del garaje del dinero del jugador. Si el jugador no tiene dinero, pierde toda la partida, y deberá generar un nuevo *savefile*.

Para persistir las partidas, cada jugador ingresará su nombre, y se guardará su dinero junto con su garaje. El *savefile* debe tener en el nombre del archivo el nombre del jugador, de alguna forma.

¡El jugador deberá reservar sus mejores vehículos para completar las misiones más difíciles y, así, sobrevivir en la ciudad!

Apartado visual

El programa deberá contar con una clara representación del mapa de la ciudad, incluyendo las calles transitables, los edificios, puntos de salida y destino, recompensas y el camino óptimo del GPS.

La implementación es libre y queda a elección del grupo. Pueden usar bibliotecas gráficas, como **JavaFX**. Una opción es tener una ventana que se actualice en cada movimiento, mostrando la información necesaria.

Otra opción es hacer una CLI (*Command Line Interface*), utilizando códigos de escape ANSI para cambiar el **color** de los caracteres y/o el fondo, o usando **emojis** compatibles con UTF-8.

Recomendación: empezar con una CLI sencilla, independiente del resto del código del proyecto, y cambiar a una librería gráfica hacia el final del trabajo, en caso de tener tiempo suficiente para su implementación en el proyecto.

Pruebas

El trabajo contará con pruebas automatizadas para una serie de **TDAs** que deberán implementar. Estos **TDAs** son los que luego deberán utilizar para desarrollar el programa, salvo excepciones que sean aclaradas explícitamente.

No se pueden utilizar las estructuras de datos de *java.util*, excepto **HashMap** y **HashSet** en la clase **Grafo** (provista por la cátedra).

Informe de complejidad algorítmica

Deberán analizar experimentalmente la complejidad del algoritmo **A*** y por lo menos **dos** algoritmos más para implementar el GPS del mapa y luego elegir, en base a algún criterio, una de las implementaciones para su juego. Para esto, comparen el tiempo de ejecución y la cantidad de operaciones para cada uno de los algoritmos bajo las mismas condiciones (mismo grafo).

Para asegurar que las condiciones son las mismas entre diferentes ejecuciones del programa, utilicen una semilla en los generadores aleatorios.

Es recomendable escribir un programa principal aparte para este informe. Implementen los algoritmos de la forma más genérica posible, para luego pasar cualquier mapa.

Finalmente comparen los resultados y escriban conclusiones.

[Plantilla para el informe de complejidad algorítmica](#)

[Plantilla de Matplotlib, para hacer gráficos](#)

Criterios de corrección

Para que el trabajo tenga buena recepción, se deberán cumplir las siguientes pautas:

1. Implementar las clases de manera totalmente original y propia. La detección de copias resultará en la **pérdida de la regularidad de los involucrados**.
2. El código **debe pasar** las pruebas automatizadas brindadas por la cátedra. **No serán corregidos** trabajos que no ejecuten todas las pruebas correctamente.
3. Las pruebas automatizadas **no pueden ser alteradas** de ninguna forma.
4. El código debe estar escrito en **camelCase**. Los nombres de las clases se escriben en **PascalCase** (las iniciales de las palabras con mayúscula). Los nombres de los archivos fuente deben tener el mismo nombre que la clase. Las constantes se escriben en **mayúsculas** y en **snake_case** (por ejemplo, UNA_CONSTANTE).
5. Es recomendable utilizar un *formatter*. Pueden utilizar el [estilo de código de Google para Java](#).

Los criterios de evaluación y corrección por parte de la cátedra son:

1. Funcionalidad:

- a. El programa debe pasar las pruebas automatizadas.
- b. El programa debe funcionar correctamente y sin errores durante la ejecución.

2. Estilo de código:

- a. **Uso del paradigma:** se debe aplicar **POO**, creando las clases que sean necesarias.
- b. **Eficiencia espacial:** el uso de memoria debe ser apropiado, tanto en términos de variables/atributos como de *tamaño en bytes* de los tipos de datos utilizados.
- c. **Eficiencia temporal:** la complejidad de sus métodos debe ser apropiada. Los algoritmos deben utilizar estructuras de datos apropiadas.
- d. **Modularización:** tanto de código en métodos como de funcionalidades en clases.
- e. **Documentación:** se deberá documentar claramente las pre/postcondiciones de sus métodos. Pueden usar *Javadoc*.
- f. **Uso de archivos:** la escritura/lectura de archivos debe ser apropiada, minimizando las aperturas y cerrando correctamente los archivos.
- g. **Buenas prácticas** de programación propuestas por la cátedra.

Formato de entrega

Para realizar la entrega, deben seguir los siguientes pasos:

1. Subir el código **fuentes** de la entrega a la branch *main* del repositorio privado de GitHub. **No** se deben subir los archivos de configuración del IDE.
2. Generar un *release* sobre el commit que quieran entregar.
3. Descargar el archivo .zip del *release*. Este archivo deberá tener el formato **tpg-2c2025-(NOMBRE_GRUPO).zip**.
4. Subir el archivo .zip al campus. Solo se puede hacer **una entrega** en el campus.
5. Entrega únicamente el delegado del grupo.

Se corregirá el commit realizado sobre la branch *main*. El archivo .zip entregado en el campus solo será revisado ante casos excepcionales.

El plazo de entrega vence el día **DOMINGO 30 DE NOVIEMBRE** a las **23:59 hrs.** No se aceptarán entregas fuera de término.

Lista de entregables

El grupo deberá entregar:

1. Repositorio de GitHub: completar el **README.md** con la información pedida.
2. Informe de complejidad algorítmica.
3. **Diagramas UML** de las clases principales del proyecto. Para simplificar, modelen únicamente las clases, sin métodos ni atributos. *Recomendación:* Usar **PlantUML** o draw.io.

Para **alguno** de los siguientes TDAs, deberán realizar el diagrama **completo**:

- a. Cola de prioridad (heap)
 - b. Diccionario
 - c. Lista
4. Video de demostración del programa, de menos de **cinco minutos**, mostrando el correcto funcionamiento del programa.

Repositorio base del TP

Tendrán a disposición un repositorio base preconfigurado para el desarrollo y entrega del TP, donde encontrarán las pruebas automatizadas que validan el funcionamiento de sus TDAs. Estas pruebas podrán ejecutarlas de forma local con **JUnit** antes de entregar para verificar que cumplen con los requisitos de entrega.

Para generar un repositorio, el delegado del grupo ingresará en el siguiente enlace, y generará el grupo con el nombre elegido.

[Enlace al repositorio base del TP](#)

Una vez generado el grupo, se podrán unir los demás integrantes.

